REPRODUCTION LOG

Every Step, Exactly As It Was Done

The commands, the URLs, the amounts, the waiting times, and the things that went wrong — written so that somebody with an empty machine can arrive at the same results.

FIELD	VALUE
Project	50081 — L2 finality benchmark, Mitacs Globalink 2026
Machine	Linux, Python 3.10.12 via pyenv
Test account	`0xAEb018EdC9cA4D70D9488FF2d38F74dd81Ad3122` — testnet only, created 7 Aug 2026
Period	7 Aug 2026 (setup) to 11 Aug 2026 (analysis)
Outcome	1,265 transactions, 97.9% success, two rollups, four chains observed

NOTHING SECRET APPEARS IN THIS DOCUMENT

The private key and the Infura project ID are in `.env`, which is gitignored and is not reproduced here. Where a URL contains a key, it is shown with a `<placeholder>`. Substitute your own.

1 · Environment

Python 3.10.12 under pyenv. `.python-version` activates the environment automatically on entering the directory.

```
pyenv virtualenv 3.10.12 l2_bench
pyenv local l2_bench
pip install -r requirements.txt
```

Or without pyenv:

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

Every version is pinned with `==`, not `>=`, so the environment can be recreated exactly. The ones that matter:

PACKAGE	VERSION	NEEDED FOR
`web3`	7.16.0	Everything. Note v7 names the signed field `raw_transaction`; v6 called it `rawTransaction`
`eth-account`	0.13.7	Key handling and signing
`py-solc-x`	2.0.5	Compiling the workload token; downloads solc itself on first use
`pandas`, `matplotlib`	2.3.3, 3.10.9	Analysis and figures
`weasyprint`, `CairoSVG`	69.0, 2.9.0	Rendering these documents and the diagrams

2 · Creating the wallet

```
python -m bench.create_wallet
```

Generates one account, writes the private key to `.env` at mode `600`, and the address to `bench/configs/accounts.yaml`, which *is* committed so every result traces back to the account that produced it. It refuses to overwrite an existing key.

One account is used on every network. EVM addresses are compatible across chains, so this keeps the records simple. Verify at any time with:

```
python -m bench.core.wallet
# 0xAEb018EdC9cA4D70D9488FF2d38F74dd81Ad3122
```

That command fails loudly if the key in `.env` derives a different address from the one recorded in config — which would mean every result was untraceable.

3 · Getting testnet funds

This is the step that blocks everything else, and the one with the most friction. Faucets refuse, rate-limit and demand accounts you may not have, so the strategy is to request from several at once rather than in sequence.

Ethereum Sepolia — fund this first

Everything else is bridged from here, so it is the only faucet you strictly need.

FAUCET	URL	REQUIRES
PoW faucet (pk910)	`sepolia-faucet.pk910.de`	**Nothing.** The browser mines for a few minutes. Start here
Google Cloud Web3	`cloud.google.com/application/web3/faucet/ethereum/sepolia`	A Google account; claimable daily
Alchemy	`sepoliafaucet.com`	~0.001 ETH on Ethereum mainnet
Chainlink	`faucets.chain.link/sepolia`	A funded mainnet account

The list is in `bench/configs/networks.yaml` and can be printed without opening the file:

```
python -m bench.check_funds --faucets
```

> GAP — NEEDS FILLING IN BY THE INTERN
>
> Sepolia was funded on 7 August, before this log began, and `accounts.yaml` still records the faucet as `TODO`. The same is true of OP Sepolia, which was funded on 10 August. Which faucet paid, and when, belongs in the report's experimental setup — fill in the `faucet:` fields in `bench/configs/accounts.yaml` from memory or browser history while it is still recoverable.

Checking what actually arrived

```
python -m bench.check_funds --priority
```

Prints chain ID, current block, balance and status for each network, and then the specific next action for whatever is missing. Exit status is `0` when every required network is funded, so it can gate a script rather than being read by eye.

```
account  0xAEb018EdC9cA4D70D9488FF2d38F74dd81Ad3122   created 2026-08-07

network                          chain          block    balance (ETH)  status
-------------------------------------------------------------------------------
Ethereum Sepolia             11155111     11,458,133         0.532559  funded
zkSync Era Sepolia               300      8,084,219         0.049853  funded
OP Sepolia                   11155420     47,277,404         0.009995  funded
```

4 · Bridging to the L2s

With Sepolia funded, bridging is far easier than fighting L2 faucets, most of which demand a mainnet balance anyway.

zkSync Era Sepolia — what was actually done

1. Import the test account into MetaMask using the key from `.env`. It is a throwaway testnet key that has never held real funds, so this is safe — but do not reuse that browser profile for anything real.
2. Open `portal.zksync.io/bridge/?network=sepolia`
3. Connect, and confirm the network reads **zkSync Era Sepolia, chain 300**.
4. Bridge **0.05 ETH**.
5. Do not watch it. It takes roughly 15 minutes.

Recorded outcome. Sepolia fell from 0.532979 to 0.482559 — the 0.05 plus about 0.0004 in L1 gas. It arrived on the L2 as **0.050102**, slightly *more* than was sent, because zkSync refunds the unused portion of the L2 gas paid up front with the deposit.

How much to bridge, and why 0.05

ITEM	COST
One L2 transfer at 0.025 gwei	~0.0000005 ETH
The full ~550-transaction experiment matrix	~0.001 ETH
Deploying the ERC-20 once	~0.00013 ETH
0.05 bridged	**~50x the matrix, with headroom for a gas spike**

0.02 would have been sufficient. The only cost of an under-bridge is another 15-minute round trip in the middle of the work, so overshooting is cheap.

OP Sepolia

The official bridge at `app.optimism.io/bridge` did not work during this project. Alternatives, best first:

ROUTE	URL	NOTE
Superchain faucet	`console.optimism.io/faucet`	Gives OP Sepolia ETH **directly** — no bridging. Auth via GitHub, or holds-mainnet-ETH
Superbridge	`superbridge.app/op-sepolia`	The most reliable third-party UI for OP-stack testnets
Brid.gg	`testnet.brid.gg/op-sepolia`	Same job, different frontend

Direct deposit	OptimismPortal on Sepolia	A plain ETH transfer to the portal credits the same address on the L2. Verify the address from Optimism's docs first — sending to the wrong contract loses the funds

0.01 ETH arrived, which is ample: an OP Sepolia native transfer costs about 0.00000002 ETH.

5 · RPC endpoints

Public endpoints throttle. Phase C queries wide block ranges and meets those limits quickly, so a free provider key is worth registering before it is needed.

Alchemy or Infura? Both cover Sepolia, which is what the settlement queries need. Alchemy additionally covers zkSync Era Sepolia, OP Sepolia and Arbitrum Sepolia under one key, so a single signup can replace every public endpoint. Infura was used here and works.

Set the override in `.env` — the variable is `BENCH_RPC_` plus the network key in upper case:

```
BENCH_RPC_SEPOLIA=https://sepolia.infura.io/v3/<your-project-id>
```

> THE MISTAKE THAT COST AN HOUR
>
> The project ID was pasted on its own, without the URL around it. That produced a `MissingSchema` traceback from deep inside `requests` that said nothing about which variable was wrong. The loader now rejects a non-URL override by name, with the correct form printed. It needs the **whole endpoint**, not just the key.

Also worth knowing: on the Infura free tier, a broad `eth_getLogs` returns `service temporarily unavailable` rather than a rate-limit error. Settlement queries must stay within tight block ranges.

6 · Deploying the workload token

Two workloads are measured: a native transfer, which needs nothing, and an ERC-20 transfer, which needs a token deployed on each L2.

Getting genuine OpenZeppelin sources

Rather than writing an ERC-20, the real OpenZeppelin sources were vendored at a pinned version and committed:

```
npm pack @openzeppelin/contracts@5.0.2
tar xzf openzeppelin-contracts-5.0.2.tgz
# copy into bench/contracts/vendor/@openzeppelin/contracts/ :
#   token/ERC20/ERC20.sol
#   token/ERC20/IERC20.sol
#   token/ERC20/extensions/IERC20Metadata.sol
#   utils/Context.sol
#   interfaces/draft-IERC6093.sol
```

`bench/contracts/BenchToken.sol` is then twenty lines that inherit from it — no hooks, no pausing, no permit. Anything unusual in the contract would appear in the gas and DA figures as though it were a property of the rollup.

Deploying

```
python -m bench.deploy_token -n zksync_sepolia --dry-run   # will the chain take it?
python -m bench.deploy_token -n zksync_sepolia --record    # deploy, write the address
```

`--dry-run` asks the node to estimate the deployment. If the estimate succeeds, the chain accepts the bytecode. This is how the question "does zkSync need its own compiler?" was answered by measurement rather than assumption.

Result: zkSync Era Sepolia *does* accept ordinary EVM bytecode — its EVM interpreter is enabled — so no `zksolc` toolchain was needed. Both deployments landed at the *same address*, `0xA1b9fdb1599fd89250B3463A1daA96c32C4BD958`, because contract addresses derive from sender plus nonce and both were nonce 0 on their chain. That is a coincidence, not a design.

NETWORK	GAS ESTIMATED	GAS USED	COST
zkSync Era Sepolia	5,386,132	590,512	0.00013465 ETH
OP Sepolia	555,635	550,102	0.00000056 ETH

zkSync over-estimated by a factor of nine; OP was accurate. This is why every cost figure in the study comes from a receipt and never from an estimate.

7 · Finding the L1 contracts

Two of the three timestamps are read from Ethereum, so the rollup's L1 contracts must be located. Each was found differently, and one of them was found wrongly first.

zkSync Era Sepolia

Asked the chain itself rather than copying an address from documentation:

```
w3.provider.make_request("zks_getMainContract", [])
# 0x9a6de0f62aa270a8bcb1e2610078650d539b1ef9
```

> THE ADDRESS THAT LOOKED RIGHT AND WAS NOT
>
> The first candidate was `0xedf4B1C6AF6520bb76826525adef7e0e09Af2c8d`, taken from the `to` field of an actual batch-posting transaction — which seems authoritative until every getter on it reverts. That address is the **ValidatorTimelock**: batches are submitted *through* it, so it is what the commit transaction points at, but it holds no batch state. A log query aimed there finds nothing and looks like a broken query rather than a wrong address. Both are now recorded in config.

The ABI came from Matter Labs' published package, because Etherscan's V1 API is retired and V2 requires a key:

```
npm pack zksync-ethers@6
# package/abi/IZkSyncHyperchain.json  ->  bench/abis/zksync_hyperchain.json
```

OP Sepolia

The batcher address is stored on the L2 itself, at the `L1Block` predeploy `0x4200000000000000000000000000000000000015`. Reading `batcherHash()` gives `0x8f23BB38F531600e5d8FDDaAEC41F13FaB46E98c`. Scanning Sepolia for that address's transactions showed them all going to one inbox, `0xff0000000000000000000000000000011155420` — which matches the superchain registry, a useful independent confirmation.

The remaining addresses came from the registry:

```
https://raw.githubusercontent.com/ethereum-optimism/superchain-registry/main/superchain/configs/
sepolia/op.toml
```

CONTRACT	ADDRESS ON SEPOLIA
Batch inbox	`0xff0000000000000000000000000000011155420`
Batcher (EOA)	`0x8f23BB38F531600e5d8FDDaAEC41F13FaB46E98c`

DisputeGameFactory	`0x05F9613aDB30026FFd634f38e5C4dFd30a197Fa1`
OptimismPortal	`0x16Fc5058F25648194471939df75CF27A2fdC48BC`

8 · Running experiments

Before the first run

```
python -m bench.check_workloads --all     # gas-estimate both workloads, send nothing
python -m bench.fetch_price               # record the ETH/USD rate with its sources
```

A single run

```
python -m bench.submit_run -n zksync_sepolia -w native_transfer -c 50
```

Sends 50 transactions, records t_0 and t_1, writes `bench/results/<run_id>.jsonl`, and exits within minutes. It never waits for settlement.

Filling in settlement

```
python -m bench.resolve_run          # one pass
python -m bench.resolve_run --watch 300   # repeat until nothing is outstanding
```

Safe to run repeatedly and safe to kill. Settlement took roughly 25 minutes to t_2 and 37 minutes to t_3 on zkSync Sepolia; on OP Sepolia t_3 is a computed deadline seven days out and will never be observed.

The matrix

```
python -m bench.run_matrix --plan     # the checklist
python -m bench.run_matrix --next     # run the next due cell, then stop
python -m bench.run_matrix --paired -w native_transfer -c 50   # both rollups, back to back
```

Unattended

```
crontab -e
# then add:
17 * * * * /home/unk/projects/l2_benchmarking/bench/collect.sh
```

Each tick submits at most one overdue cell, resolves what has settled, checks the invariants, and exits. The 60-minute gap between repetitions of one cell is enforced inside `run_matrix`, not by cron, so a tick that finds nothing due simply exits.

9 · Analysis

```
python -m bench.export                    # raw CSV, summary, assumptions, comparison
python -m bench.analysis.figures          # the four figures, each with a CSV
python -m bench.analysis.reproducibility  # day-to-day variation
python -m bench.check_data                # are the rows possible?
python -m bench.check_config              # does every setting do something?
python -m bench.observe_mainnet           # production cadence, read-only
python docs/build_handbook.py             # regenerate these documents
```

10 · Everything that went wrong

Each of these cost real time. They are the most useful part of this document.

SYMPTOM	CAUSE	FIX
`MissingSchema` from inside requests	Infura project ID pasted without the URL around it	Use the full endpoint. The loader now rejects a non-URL override by name
`execution reverted`, no reason, only on zkSync	Transfers to `0x…dEaD`. That address is 57,005, inside the range zkSync reserves for system contracts	Use a recipient above ~0x20000. Estimates fine on Sepolia, fails only on the rollup
`gas required exceeds allowance (1)`	A placeholder `gas: 1` left on the dict passed to `estimate_gas`. Geth reads it as the simulation allowance; zkSync ignores it	Strip `gas` before estimating
Every getter on the L1 contract reverts	Using the ValidatorTimelock instead of the diamond proxy	`zks_getMainContract`
t_3 null for hours on zkSync	`zks_getTransactionDetails` lags the per-batch view by tens of minutes	Read settlement per batch via `zks_getL1BatchDetails`
t_2 nine seconds *before* t_0	Scanning for OP's batch from the L2 block's L1 origin, which lags the head by a sequencer window	Anchor the scan by timestamp; refuse any settlement time earlier than t_0
196 rows with t_3 209 days in the past	Binary-searching the dispute game factory, which is **not** ordered by L2 block, and not filtering by `gameType`	Scan backwards from the newest game, filtered to the portal's `respectedGameType`

HTTP 429 mid-batch	Two submitters against one public endpoint	Retry reads with backoff. **Never** retry `eth_sendRawTransaction`
Every dollar figure roughly double	The ETH rate was a hand-typed `3800.0` placeholder; the real rate was 1,878.57	Fetch it, record the sources and the timestamp. Prices apply at analysis time, so this was a re-export, not a re-run
A network that answers correctly but has no recent blocks	The public Polygon zkEVM Cardona endpoint was serving state 38 days old	Compare the head block's timestamp against the wall clock

11 · Things that cannot be measured this way

Recording these saves somebody else the same discovery.

- **Per-transaction data-availability size.** A blob costs 131,072 bytes whether the rollup fills it or not, and the batch carries every user's transactions. At these volumes DA per transaction is a batch average that does not vary with workload.
- **Cost as a function of *our* batch size.** Seven runs of sizes 1 to 50 all landed in one batch of 875 and cost exactly the same per transaction. The denominator is the rollup's batch count, which we do not control. The rollup's own variation does answer it: across eight batches from 617 to 1,737 transactions, cost per transaction falls 66%.
- **Observed trustless finality on an optimistic rollup.** OP Sepolia's `proofMaturityDelaySeconds` is 604,800 — a full seven days, identical to OP Mainnet. Testnets are widely assumed to shorten this. This one does not, so t_3 there is always a computed deadline.
- **Peak throughput.** Saturating a shared public sequencer would degrade a service other people depend on, and would measure our own rate limits rather than the system.

12 · Timeline as it actually happened

WHEN	WHAT
7 Aug, 15:49	Repository, wallet, network config, funding checker (A1–A4)
7 Aug, 17:44	First transaction sent by hand on Sepolia (A5)
10 Aug, 13:01	Workloads defined, token deployed on zkSync (B1)

10 Aug, 13:23	Submission, nonces, t_1, failure handling (B2–B5)
10 Aug, 13:49	OP Sepolia funded, its token deployed
10 Aug, 14:01	Settlement mapping and t_2 (C1–C6)
10 Aug, 14:15	Export, real L1 costs, DA, throughput (D1–D5) — minimum viable deliverable
10 Aug, 15:05	Optimistic rollup and challenge-window model (E1, E2)
10 Aug, 15:21	Experiment matrix, paired run, mainnet observation (F1, F4, E3)
10 Aug, 15:29	Figures (G1–G3)
10 Aug, 15:38	Hourly collection scheduled (F2 begins)
11 Aug, 08:30	Batch scaling and reproducibility after 18 unattended ticks (F3, G4)
11 Aug, 08:36	ETH rate corrected; every cost figure re-derived

Roughly two working days of hands-on effort, plus overnight collection. The long pole was never the code — it was waiting for settlement, which is precisely why submission and resolution were separated.